

A Combined Streaming FPGA Architecture for Client-Side CKKS Encoding and Encryption

Joseph Kurina

University of Washington
Bothell, Washington, USA
kurinj@uw.edu

Bogdan Pasca

Altera Corporation
Paris, France
bogdan.pasca@altera.com

Suleyman Demirsoy

Altera Corporation
London, U.K.
suleyman.demirsoy@altera.com

Sunwoong Kim

Rochester Institute of Technology
Rochester, New York, USA
sskeme@rit.edu

Abstract—Homomorphic encryption allows a computing service to process encrypted data, but the client must first encode and encrypt its inputs. Sharing hardware between floating-point encoding and modular encryption reduces area but requires those transform roles to execute at different times. We present a streaming FPGA architecture for CKKS encoding and encryption that computes the shared floating-point encoding transform once and performs the integer transforms for six residue moduli in parallel. Dedicated transform pipelines exchange intermediate results on chip rather than through off-chip memory. DSP Builder generates the arithmetic engines, while SYCL kernels express the surrounding computation and connect the transforms through a pipelined architecture. At polynomial degree $N = 8192$, the Altera Agilex 7 implementation delivers 980 complete ciphertexts per second, compared with 113.2 for one SEAL-Embedded worker on the same workstation, giving $8.66\times$ the throughput. Measurements include encoding and encryption arithmetic, FPGA submission, output delivery, and ciphertext construction. Inputs are prepared before timing begins.

Index Terms—CKKS, homomorphic encryption, FPGA, SYCL, DSP Builder, streaming architecture, IFFT, NTT.

I. INTRODUCTION

Homomorphic encryption (HE) allows a computing service to operate on encrypted data without decrypting it [1]. Applications include privacy-preserving neural-network inference [2], image processing [3], and speech recognition [4]. The data owner retains the secret key, prepares encrypted inputs, and decrypts the returned results. The Cheon, Kim, Kim, and Song (CKKS) scheme supports approximate arithmetic on packed real or complex values [5]. Packing places many values in one ciphertext so that the service can process them together.

Preparing that ciphertext requires substantial computation before the service can begin. The client maps a message vector to polynomial coefficients using a complex-valued transform, scales and rounds them to integers, and encrypts the polynomial. A residue number system (RNS) represents the large ciphertext modulus through several smaller primes, requiring modular arithmetic for every prime [6]. Client-side encoding and encryption therefore combine floating-point and integer workloads, whereas most existing HE hardware research addresses server-side ciphertext operations [7]–[14].

Existing client accelerators balance arithmetic reuse against concurrent execution. Aloha-HE reuses butterfly units to reduce area, while ABC-FHE uses parallel streaming cores with shared Fourier and modular transform arithmetic [15], [16].

The accelerator proposed in this paper uses a workstation-attached FPGA with dedicated parallel number theoretic transforms (NTTs) and shares only the modulus-independent inverse fast Fourier transform (IFFT). This partition removes competition for transform engines between moduli and between the secret-key and plaintext-plus-error paths. The resulting architecture trades arithmetic reuse for concurrent encoding and encryption. Encoded coefficients pass into encryption through on-chip buffers, avoiding an intermediate off-chip memory transfer. Here, *client-side* identifies the data-owner operation performed by the workstation, which supplies parameters, keys, and randomness, and assembles the ciphertext.

The integration challenge is to preserve coefficient order across transforms and prevent input or output storage from being overwritten while still in use. SYCL is a C++ programming model in which host code submits device kernels and tracks their completion through events. OneAPI FPGA extensions provide typed on-chip FIFOs called pipes and calls to imported register-transfer-level (RTL) components [17]. We use these mechanisms to connect RTL developed with Altera’s DSP Builder HLS to SYCL polynomial-arithmetic kernels and host-side encryption libraries. Our contribution is the combined pipelined encryption engine and library integration interface: import buffers store transform outputs, FIFO pipes carry ordered coefficients, and host events enable efficient buffer reuse. Each transform transfers four coefficients together through four parallel data paths, called *coefficient lanes*. These lanes carry parts of one polynomial. They do not encrypt four messages independently.

The main contributions are:

1. A streaming symmetric CKKS architecture at $N = 8192$ with one four-coefficient-lane IFFT using complex doubles and six physical 30-bit RNS pipelines, each with two dedicated NTT engines.
2. A hybrid SYCL/RTL implementation that buffers and pipelines the outputs of RTL transforms, connects persistent arithmetic kernels, and returns complete ciphertexts via reusable unified shared memory (USM) allocations.
3. An evaluation against one native SEAL-Embedded [18] worker running on the same host CPU, which shows our FPGA-based design achieves an $8.66\times$ speedup over the SEAL-Embedded CPU implementation when processing batches of four encoding-and-encryption operations.

The accelerator project repository is https://github.com/joekurina/FPT26_Agilex7_CKKS_Accelerator.

II. BACKGROUND AND RELATED WORK

A. CKKS Encoding and Symmetric Encryption

CKKS represents plaintexts and ciphertext components in the negacyclic ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$, where N is a power of two and $Q = \prod_i q_i$ is a product of pairwise-coprime primes. Polynomial coefficients are integers modulo Q . Multiplication wraps powers of X^N to -1 . RNS replaces arithmetic modulo the large Q with independent computations modulo each q_i [6]. We call each polynomial's representation modulo one prime an RNS *limb*.

Encoding takes $N/2$ complex plaintext values, called slots, and expands them into a conjugate-symmetric vector of length N . A negacyclic inverse transform maps this vector to polynomial coefficients. Scaling by Δ and rounding produces an encoded integer plaintext polynomial m , which is reduced modulo each prime [5], [6]. The scale controls the representation of fractional values.

Symmetric encryption combines m with a small error polynomial e , the secret-key polynomial s , and a uniform polynomial a sampled in NTT representation. The NTT is the modular counterpart of the Fourier transform. It enables polynomial multiplication to be performed by efficient point-wise multiplication of coefficients. With i indexing the RNS modulus, define $\widehat{m+e}_i = \text{NTT}_{q_i}(m+e)$ and $\widehat{s}_i = \text{NTT}_{q_i}(s)$. The ciphertext components in NTT representation are

$$c_{1,i} = a_i, \quad (1)$$

$$c_{0,i} = \widehat{m+e}_i - \widehat{s}_i \odot a_i \pmod{q_i}, \quad (2)$$

where $\odot$ denotes point-wise coefficient multiplication. All per-modulus pairs $(c_{0,i}, c_{1,i})$ together represent one ciphertext $(c_0, c_1) \in R_Q^2$. The two NTTs in (2) can run independently, and each modulus admits a separate computation.

Both transforms use $O(N \log N)$ butterfly arithmetic, where each butterfly combines a small group of coefficients. The IFFT operates on complex doubles, whose real and imaginary components are each stored as a 64-bit double-precision floating-point value. The NTT operates in $\mathbb{Z}_{q_i}$ and uses a primitive $2N$ -th root of unity for the negacyclic transform [19]. At $N = 8192$, each radix-2 transform has 13 stages. Our configuration uses six 30-bit primes satisfying $q_i \equiv 1 \pmod{2N}$. Modular multiplication outside the imported NTT uses Barrett reduction [20].

B. Related Work

Microsoft SEAL implements CKKS in software [21]. SEAL-Embedded, the baseline used here, reduces the memory requirements of client-side CKKS operations for embedded processors [18]. Hardware clients include CHOCO-TACO for encrypted compute offloading [22], RACE and RISE for RISC-V systems [23], [24], and the NTT accelerator integrated with SEAL-Embedded by Di Matteo *et al.* [25]. Software and protocol approaches also reduce client work by exploiting

TABLE I
DESIGN PARAMETERS OF THE PROPOSED ARCHITECTURE

Parameter	Value
Polynomial degree N	8,192
Complex slots	up to 4,096
Transform stages	13
Coefficients per transfer	4
Transfers per polynomial	2,048
RNS moduli / physical modulus pipelines	6 / 6
Modulus width	30 bits
IFFT arithmetic	Complex double precision
NTT and ciphertext residues	32 bits
NTT engines	12

encoding redundancy [26] or using a bootstrapping-assisted client protocol [27].

Client accelerators make different choices about supported operations and arithmetic reuse. Lee *et al.* provide configurable CKKS encryption and decryption architectures [28]. Aloha-HE shares one double-precision FFT butterfly and three modular NTT butterflies across encoding, encryption, decryption, and decoding to reduce area [15]. ABC-FHE uses two reconfigurable streaming cores that share arithmetic between FFT and NTT modes [16]. Section IV-E compares these designs with our dedicated-transform partition and distinguishes their measurement boundaries.

HEXL-FPGA implements HE arithmetic primitives in SYCL, including NTT and key switching, rather than focusing on client-side encoding and encryption [29]. Our design combines DSP Builder-generated RTL transforms with SYCL kernels for these client operations.

III. ACCELERATOR ARCHITECTURE

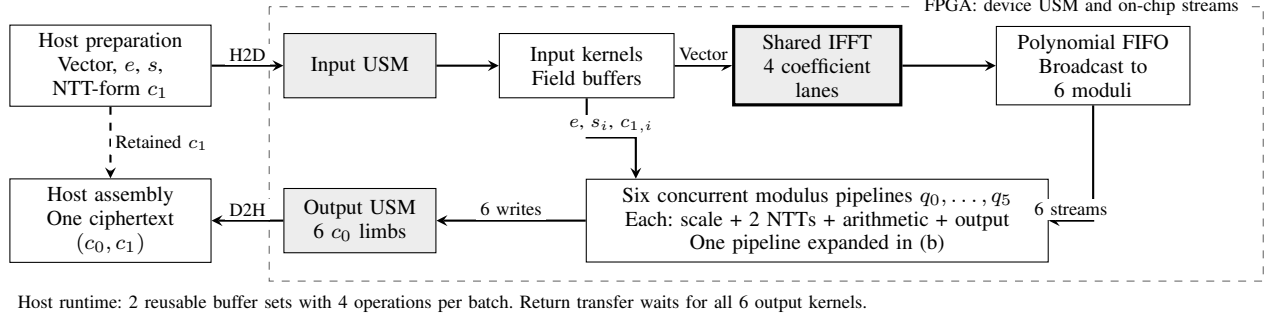
This section presents the proposed accelerator architecture for the CKKS client operations. Table I summarizes the fixed configuration.

A. Overview and Design Parameters

Fig. 1 shows the complete dataflow and expands one of the modulus pipelines. One operation consumes a message vector and produces one complete ciphertext. In the evaluated runtime, a *batch* groups four such operations for host submission. Within an operation, each transform accepts four coefficients per transfer: these are the four *coefficient lanes*. The six *modulus pipelines*, by contrast, process six representations of the same polynomial, one for each prime.

In Fig. 1(a), input kernels read the encoding vector and auxiliary data from device memory. The encoding vector traverses the shared IFFT. Error samples, secret-key coefficients, and NTT-form c_1 bypass it. A polynomial buffer broadcasts the IFFT result to the six modulus pipelines. Each pipeline scales and reduces the coefficients, computes the two NTTs, and forms its c_0 limb as expanded in Fig. 1(b). Dedicated engines let the modulus computations proceed without sharing a transform scheduler. Six output kernels write contiguous c_0 limb regions, which return to the host in one bulk transfer. The host combines them with retained c_1 .

(a) Complete host-FPGA dataflow



(b) One physical modulus pipeline q_i (six spatial copies)

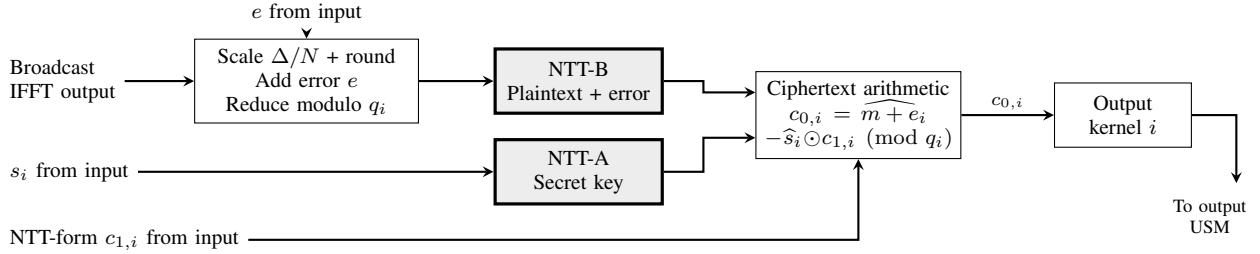


Fig. 1. CKKS encoding-and-encryption dataflow. (a) A shared IFFT feeds six concurrent modulus pipelines. Auxiliary streams bypass it. The host combines six computed c_0 limbs with retained c_1 into one ciphertext. (b) Each modulus has independent NTT-A and NTT-B engines. Streams carry four coefficients per transfer, with complete-vector buffers aligning consumers.

B. SYCL and the Hardware/Software Partition

DSP Builder generates the computationally intensive IFFT and NTT datapaths as RTL cores [30]. We package these cores as imported libraries and call them from SYCL device kernels. Other SYCL kernels distribute input, broadcast coefficients, apply scaling and reduction, compute ciphertext arithmetic, and collect output. Pipes carry intermediate coefficient groups between these kernels on chip.

The host-side C++ program uses SEAL-Embedded to obtain encryption parameters, keys, and randomness, submits prepared inputs to the FPGA, and constructs ciphertext objects from the returned results. USM provides allocations addressable through C++ pointers. Explicit host/device transfers move the input and output payloads. Queue events track when kernels and transfers finish [17]. This division retains specialized RTL arithmetic while expressing system connections and buffer lifetimes in C++. The RTL interfaces must preserve coefficient order and valid-data signaling, and the runtime must wait for readers before reusing their storage.

C. Input Buffering and Shared IFFT

The input stage separates device-memory loading from delivery to the arithmetic pipelines. The input is double-buffered across two banks. Each bank stores a complete operation’s encoding vector, shared error samples, and per-modulus secret-key and c_1 coefficients. Input kernels then send each field to its consumer in coefficient groups, preserving operation order even when the consumers progress at different

rates. A bank becomes free for a new input vector only after every destination has accepted its final group. Thus, a stalled consumer cannot cause unread input to be overwritten.

The imported IFFT RTL IP core is based on a Gentleman–Sande decimation-in-frequency butterfly:

$$u = a + b, \quad (3)$$

$$v = (a - b) \cdot \omega, \quad (4)$$

where a , b , and ω are complex doubles. The four coefficient lanes carry one group of four complex values per accepted transfer, so a length- N vector requires 2,048 transfers.

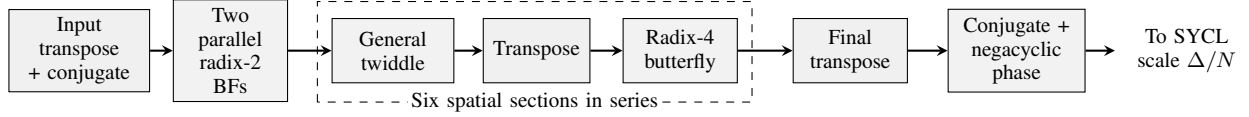
A 2,048-entry, 512-bit FIFO buffer holds a complete IFFT output vector. The broadcast kernel reads each four-value group once and delivers a copy to every modulus pipeline before advancing. This buffer captures the output of the IFFT and accommodates short-term differences in consumer progress.

D. DSP Builder Streaming Transform Architecture

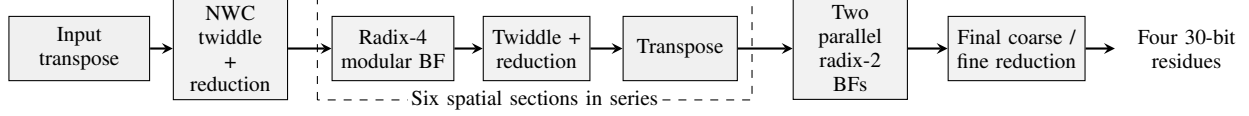
Fig. 2 compares the IFFT and NTT pipelines and expands the transpose-memory mechanism that aligns operands between butterfly sections.

DSP Builder generates pipelines of butterfly sections, twiddle-factor arithmetic, and memory-based permutations. The 13 radix-2 stages are grouped into seven mixed-radix sections using $8192 = 2 \cdot 4^6$. Interstage routing supplies the operands needed by each section while keeping intermediate polynomials in on-chip storage.

(a) Negacyclic IFFT: four complex doubles per transfer



(b) Negacyclic NTT: four coefficients per transfer, with the modulus selector aligned with data



(c) Transpose structure: permutation across lanes and transfer times

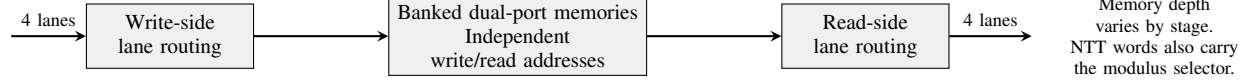


Fig. 2. DSP Builder IFFT and NTT pipelines and their transpose memories. BF denotes butterfly and NWC denotes negacyclic wraparound convolution. Each dashed group represents six spatial sections connected in series. The IFFT places radix-2 first and applies the negacyclic phase after the Fourier pipeline. The NTT places its NWC layer before the butterflies and radix-2 last. Boxes summarize pipelined operators rather than individual cycles. Panel (c) routes coefficients across lanes and transfer times. Memory widths and depths vary by stage. Import buffering is described in Section III-F.

1) *Negacyclic IFFT*: The IFFT begins with an input transpose and complex conjugation. Two parallel radix-2 butterflies feed six four-point butterfly sections, connected by general-twiddle blocks and transpose networks. A final transpose reorders the result. Within a four-point section, complex sums, differences, and fixed rotations implement the butterfly. The coefficient tables supply the general-twiddle multiplications between sections. All real and imaginary components use double-precision arithmetic. Conjugation at the output completes the unnormalized inverse transform, followed by a lane-specific, ROM-driven phase multiplication for the negacyclic transform. The SYCL stage then applies Δ/N .

2) *Negacyclic NTT*: The NTT begins with a transpose and a negacyclic twiddle multiplication/reduction layer. Six radix-4 butterfly sections alternate with general-twiddle and transpose blocks, followed by two parallel radix-2 butterflies. A modulus selector travels with the data and selects the twiddle factors and reduction constants for q_i . The butterfly adders and subtractors form integer sums and differences of their input coefficients. They retain extra bits for intermediate results, reduction around the twiddle multiplications limits their range, and final coarse/fine correction produces residues in $[0, q_i)$. The generated structure supports all six configured primes and is instantiated for each transform role in each modulus pipeline.

The transpose networks supply coefficients that arrive on different lanes or at different times to the same butterfly section. Lane-selection multiplexers reorder values within a transfer. Independently addressed dual-port memories hold earlier coefficients and read them with later arrivals in the order required by the next section. This storage implements permutations that cannot be completed by exchanging the four values of one transfer alone. Valid-data control and the NTT modulus selector remain aligned with the reordered

coefficients.

E. Parallel Modulus Pipelines

Let j index a coefficient, x_j denote an IFFT output, and e_j denote the shared error sample. For modulus q_i , scaling and reduction produce the plaintext-plus-error coefficient $p_{i,j}$:

$$t_j = \text{round}\left(\text{Re}(x_j) \cdot \frac{\Delta}{N}\right) + e_j, \quad (5)$$

$$p_{i,j} = t_j \pmod{q_i}. \quad (6)$$

Signed Barrett reduction maps t_j to $[0, q_i)$. NTT-B transforms the resulting plaintext-plus-error stream, while NTT-A independently transforms the secret-key stream. Their outputs and the supplied $c_{1,i}$ are buffered until corresponding coefficient groups can be combined.

Using the NTT notation of (2), the ciphertext arithmetic kernel computes

$$c_{0,i,j} = \widehat{m} + e_{i,j} - \widehat{s}_{i,j} c_{1,i,j} \pmod{q_i}. \quad (7)$$

Thus $c_{0,i}$ in Fig. 1 denotes the whole limb for modulus i , and $c_{0,i,j}$ in (7) denotes its coefficient j . All six modulus pipelines have separate NTT-A and NTT-B engines, giving twelve NTT engines in total. Complete-vector buffers align their outputs, and each pipeline writes its limb through an independent output kernel.

F. RTL Integration and Flow Control

The imported transforms produce output that their downstream consumers may not immediately accept. A wrapper between each RTL core and its SYCL kernel packs coefficient groups, marks valid data, and buffers results until the next stage is ready. The C++ declaration and RTL import specification agree on the placement of the four coefficients and their control fields.

1) *Persistent Kernel Operation*: At session initialization, the host launches persistent single-task kernels for transforms, broadcast, scaling and reduction, and ciphertext arithmetic. These kernels remain active for the session, processing successive coefficient streams without relaunch. Finite input and output kernels serve successive batches. Events coordinate the finite invocations and memory transfers.

The IFFT wrapper supplies four complex values when input is available. Valid results enter the output FIFO. The NTT wrappers instead wait for a coefficient group before calling the imported RTL, avoiding idle interface transactions. In modulus pipeline i , both NTT engines use q_i for every coefficient. A selector carried with the coefficients chooses that prime's twiddle factors and reduction constants. Each of the six pipelines uses a distinct prime.

2) *Burst Capture and Output-Space Reservation*: Each input wrapper has a 4,096-entry block-memory output FIFO, enough for two transforms of 2,048 coefficient groups each. This FIFO is separate from the complete-vector buffers between SYCL kernels. A nonempty FIFO presents its first entry until the downstream interface accepts it. Counters track accepted input groups and fully drained outputs to reserve space for results still in flight.

When the next stage stalls, the wrapper holds buffered output until it is accepted. It does not rely on directly stopping the generated arithmetic. The compiler treats each imported component as a variable-latency interface. Actual transfers follow the valid/ready signals rather than a fixed cycle count. Pipes coordinate these on-chip transfers, whereas host events track completed kernels and memory transfers. Output kernels can consume a batch while its input kernels are still supplying coefficients. The return transfer waits for all six output kernels.

G. Host Runtime and Ciphertext Assembly

The host alternates between two reusable sets of input/output buffers. Each set holds a batch of four operations and records which kernels and transfers have finished. The runtime can submit two batches before collecting results, but a buffer set cannot be reused until its previous batch is complete. These host/device allocations are separate from the two on-chip input banks, which alternate between individual operations within a batch.

A batch's input kernels wait for its input transfer and the preceding batch's input/output kernels. Each output kernel waits only for its previous invocation, allowing it to consume incoming coefficients while the batch is being supplied. After all six outputs finish, the runtime transfers c_0 to the host. It also checks every finite input kernel before releasing the buffers. The enabled completion latch remembers successful checks, avoiding repeated queries for kernels already known to have finished. Collection combines the returned c_0 limbs with retained host c_1 to construct complete ciphertexts. One set of finite kernel submissions serves four operations, spreading submission overhead across the batch. The host API admits up to 16 operations per batch, but this evaluation qualifies four.

TABLE II
RNS MODULI AT $N = 8192$ [31]

Pipeline i	Prime q_i
0	1,053,818,881
1	1,054,015,489
2	1,054,212,097
3	1,055,260,673
4	1,056,178,177
5	1,056,440,321

IV. EXPERIMENTAL RESULTS

We measure two different quantities. *Throughput* is the number of complete ciphertexts returned per second over a long sequence, including the host work required to submit computation and construct outputs. *Device latency* is the elapsed time for one operation between two points inside the FPGA. Throughput measures sustained input production for the application. Device latency measures the interval between hardware markers, with endpoints defined in Section IV-D. The throughput build has no added device timer. A separately compiled build adds timing logic, so its resources and latency are reported separately rather than combined with the throughput result.

A. Experimental Setup

The accelerator runs on a BittWare IA-840f card with an Altera Agilex 7 AGFB027R25A2E2V FPGA and 2×32 GB DDR4 memory. The CPU baseline is one native SEAL-Embedded worker on the same workstation: an Intel Xeon W-2295 with 128 GB DDR4 running Rocky Linux 9.7.

Both implementations use the same 16-entry corpus of full-real, 4,096-slot vectors, keys, and seeds, with $N = 8192$ and scale 2^{25} . Table II lists the six fixed primes in pipeline order, which are taken from the SEAL-Embedded open-source code [31]. The real values are float-representable and promoted to double. Both paths receive prepared embeddings, sampled errors, uniform polynomials, and expanded coefficient-domain keys. The CPU embedding is taken before the FPGA-specific conjugation step. Input and seed digests check shared input identities. Numerical output checks are described separately.

For the throughput comparison, both implementations start with inputs prepared in memory. The host timer includes IFFT, scaling and rounding, error addition, RNS reduction, both NTTs, ciphertext arithmetic, and complete ciphertext construction. FPGA timing also includes required submission, completion, and output delivery. Shared input preparation, pool allocation, warmup, validation, hashing, logging, and teardown are excluded. The native adapter preserves stock arithmetic and copies c_1 to dedicated external storage. The FPGA runtime retains host c_1 . Thus, the comparison uses the same arithmetic and output requirements, with each implementation's required data movement, but excludes online input preparation.

Before warmup, both drivers allocate and touch their output buffers. Five buffers hold five CPU ciphertexts (5.625 MiB) or five FPGA batches totaling twenty ciphertexts (22.5 MiB).

TABLE III
POST-FIT RESOURCES OF THE BUILD WITH TIMING LOGIC

Metric	Result
Nominal kernel frequency	481 MHz
ALMs	476,479
ALUTs	689,157
Registers	1,340,508
Block-memory bits	92,940,824
RAM blocks	5,202
DSP blocks	2,384

The FPGA runtime adds 12.75 MiB of payload. During timing, the drivers retain first, middle, and last outputs by exchanging buffer ownership rather than copying ciphertexts. The retained FPGA outputs come from distinct batches. Ciphertext construction remains timed. Stock-SEAL decryption and decoding are checked afterward.

Fresh 32- and 64-record qualifications precede measurements on both implementations. A record denotes one encoding-and-encryption operation. At each measured length, 1,024 or 8,192 records, each implementation runs in three fresh processes. In repeats one and three, the FPGA trial runs first and the CPU trial second. Repeat two reverses that order. Every process completes 64 warmup records before timing. All trials are retained, with no failures, timeouts, or retries. The reported rate divides total delivered ciphertexts by total measured seconds across the three repeats. One ciphertext includes both components across all six moduli.

B. Resource Utilization and Correctness

Table III reports adaptive logic modules (ALMs), adaptive lookup tables (ALUTs), registers, memory, and digital signal processing (DSP) blocks for the build with timing logic. Its nominal clock setting is 481 MHz. These figures describe that complete compiled design, including the timer. The fitter’s memory-bit usage and occupied RAM-block count are retained as distinct counters, rather than converting one into the other. The campaign did not retain a corresponding post-fit report for the throughput build, so its resource usage remains unreported.

The two qualification lengths on both backends verify 192 ciphertext executions, covering all 16 entries in each qualification. Measured runs deliver 55,296 ciphertexts, with 36 predetermined outputs numerically checked after timing. Every checked ciphertext passes numerical checks on all 4,096 decoded slots. Its complete c_0/c_1 residues lie in the required ranges, retained c_1 matches exactly, and ciphertext construction and metadata checks pass. Maximum absolute decoded error is 3.18×10^{-5} , below the 0.1 acceptance threshold.

Numerical coverage excludes warmup outputs and unsampled measured outputs. The c_0 checks use canonicity and stock-SEAL decrypt/decode.

C. Throughput Comparison with SEAL-Embedded

At 8,192 records per run, Table IV shows 980 ciphertexts/s for the FPGA and 113.2 for the CPU worker, an $8.66\times$ ratio. At 1,024 records, the rates are 972 and 113.3, or $8.58\times$. Rates

are rounded to whole ciphertexts/s for the FPGA and tenths for the CPU to reflect the observed trial variation.

D. Device Latency with Added Timing Logic

The timing build uses an on-device timer. It timestamps an input-start marker before loading and a final-result-consumption marker from each of the six output kernels. The interval ends at the last output marker and covers device input loading and computation of all six c_0 limbs for one ciphertext. It includes result-dependent signature generation before the output markers, but excludes host transfers and final host DDR memory writes. The seconds-long throughput trials instead cover thousands of complete operations and their host delivery.

Median on-device latency ranges from 80.7 to 80.8 μs at a nominal clock frequency of 481 MHz. All measured operations pass complete-ciphertext and full-slot verification with absolute decoded error of 3.29×10^{-5} . Times are rounded to tenths of a microsecond because the conversion uses a nominal clock. Table V gives the exact tick count for the representative median.

E. Comparison with Prior Custom Accelerators

Table V compares the operation scope and reported results of client custom accelerators. The following discussion relates their arithmetic organization to our dedicated encoding/encryption path.

Aloha-HE shares one FFT butterfly and three NTT butterflies across encoding, encryption, decryption, and decoding to reduce area [15]. Our one double-precision IFFT and twelve NTT engines keep the encoding/encryption transforms physically separate. This removes transform scheduling conflicts between the two NTT roles and the six moduli, at the cost of more resident arithmetic. Lee *et al.*’s configurable encryption/decryption hardware accepts an already encoded polynomial for the reported encryption rate [28]. Our pipeline also performs the preceding encoding and passes its coefficients directly to encryption on chip.

ABC-FHE already exploits parallel streaming. Its two reconfigurable cores share arithmetic between FFT and NTT modes, use custom 55-bit floating point, and generate randomness and twiddles on chip [16]. Our fixed symmetric configuration retains double-precision encoding output and dedicates each NTT to one role and modulus, with host-prepared randomness and ROM twiddles. SYCL connects those independent RTL paths to arithmetic kernels and host buffer management. ABC-FHE supports larger bootstrappable parameters and evaluates them with an ASIC cycle model. Our results measure a physical FPGA at the stated fixed configuration.

In both our design and Aloha-HE, the modulus limbs collectively represent one ciphertext. Our timer waits for the last of all six pipelines, so the measured 80.7 μs already covers their joint computation. It must not be multiplied by six, nor should six ciphertexts be counted. This device interval, Aloha-HE’s whole-system latency, Lee *et al.*’s encryption-module rate, and ABC-FHE’s cycle model have different boundaries and

TABLE IV
THROUGHPUT COMPARISON WITH SEAL-EMBEDDED

Records per run	CPU weighted ct/s	CPU trial range	FPGA weighted ct/s	FPGA trial range	FPGA / CPU
1,024	113.3	112.8–113.5	972	964–983	8.58×
8,192	113.2	113.1–113.4	980	972–988	8.66×

TABLE V
CLIENT ACCELERATOR ARCHITECTURES AND MEASUREMENT BOUNDARIES

System	Operation and parameters	Platform	Reported result	Measurement scope
This work, timing build	Dedicated IFFT/NTTs; symmetric encoding/encryption; $N = 8192$; six concurrent 30-bit moduli; full-complex input	Altera Agilex 7; nominal 481 MHz	Median 38,814 ticks; nominal 80.7 μ s	Device input loading through all six output markers; excludes host transfers and final DDR write retirement
Aloha-HE [15]	Public-key encoding/encryption; $N = 8192$, 3×54 -bit	AMD Kintex-7 / MicroBlaze; 200 MHz	1.87 ms	Whole-system latency: software, hardware, and transfers
Lee <i>et al.</i> [28]	Public-key encryption; $N = 8192$, level 5; widths {30,20,20,20,20,20,30} bits	AMD XCU250; 250 MHz	2,950 encryption operations/s	Encryption-module rate, starting with an encoded polynomial
ABC-FHE [16]	Encoding/encryption; $N = 65536$, 36-bit primes, 24 levels	28-nm ASIC design; 600 MHz	1,112× over Core i7-12700 / Lattigo	Cycle model; absolute latency shown only in a plot

parameters. They do not establish a normalized performance ranking. The measured 8.66× ratio applies only to the same-host CPU comparison in Section IV-C.

F. Discussion and Limitations

The evaluated runtime submits four operations together, alternates between two buffer sets, and checks completion before reusing either set. This configuration gives repeatable throughput when inputs are prepared beforehand. Two queued batches do not establish that their device computations overlap: dependencies still order successive input and output invocations. Stage-level timing and experiments disabling individual optimizations are needed to identify the limiting stage and attribute the gains. Application buffers are preallocated, but the measurements do not establish that the runtime avoids all further allocations. Larger batches need tests for completion and numerical correctness. Because the two timing experiments use different builds and measurement boundaries, subtracting their results would not measure host overhead.

Larger polynomial degrees require regenerated transforms, coefficient tables, permutations, and storage. Longer modulus chains could reuse the IFFT result across successive modulus groups, with additional replay storage and aligned auxiliary streams. Wider primes require revised arithmetic widths, reduction logic, and interfaces. These extensions remain implementation and validation work.

V. CONCLUSION

The accelerator connects one shared four-coefficient-lane IFFT and twelve dedicated NTT engines across six modulus pipelines through SYCL pipes. Import FIFOs capture bursts, and completion dependencies protect reusable buffers. On Agilex 7, batches of four operations achieve 980 complete ciphertexts/s versus 113.2 for one same-host CPU worker, an 8.66× ratio with input preparation excluded from both timers.

All ciphertexts in the qualification runs and the predetermined samples from measured runs pass complete-ciphertext and all-slot checks. A build with added timing logic measures a nominal 80.7 μ s median device interval. Stage-level profiling and tests of larger batches are needed to determine what limits the sustained rate. Future work will extend this architecture to CKKS parameters with larger polynomial degrees and wider RNS primes. The extension will require regenerated transforms and wider modular arithmetic, followed by correctness and performance evaluation at the new parameters.

ACKNOWLEDGMENTS

This material is based upon work supported by the National Science Foundation under Grant No. 2347253. The authors would like to thank Altera Corporation for providing the FPGA development board used in this work.

REFERENCES

- [1] J. H. Cheon *et al.*, "Introduction to Homomorphic Encryption and Schemes," Chapter in *Protecting Privacy through Homomorphic Encryption*, pp. 3-28, Springer International Publishing, 2022.
- [2] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy," in *Proc. ICML*, 2016.
- [3] S. Kim and W. Cho, "Accelerating Homomorphic Comparison Operations for Thresholding Using an Asymmetric Input Range and Input Scaling," in *Proc. GLSVLSI*, 2024.
- [4] D. L. Elworth, and S. Kim, "HEKWS: Privacy-preserving Convolutional Neural Network-based Keyword Spotting With a Ciphertext Packing Technique," in *Proc. MMSP*, 2022.
- [5] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *Proc. ASIACRYPT*, 2017.
- [6] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A Full RNS Variant of Approximate Homomorphic Encryption," in *Proc. SAC*, 2019.
- [7] W. Zhao, Q. Chen, Y. Wang, H. Zhang, Z. Lu, and G. Qu, "A Survey on FPGA-based Accelerators for CKKS," in *Proc. ITC-Asia*, 2024.
- [8] M. S. Riazi, K. Laine, B. Pelton, and W. Dai, "HEAX: An Architecture for Computing on Encrypted Data," in *Proc. ASPLOS*, 2020.
- [9] R. Agrawal, L. De Castro, G. Yang, C. Juvekar, R. Yazicigil, and A. Chandrakasan, "FAB: An FPGA-Based Accelerator for Bootstrappable Fully Homomorphic Encryption," in *Proc. HPCA*, 2023.

- [10] N. Samardzic *et al.*, "F1: A Fast and Programmable Accelerator for Fully Homomorphic Encryption," in *Proc. MICRO*, 2021.
- [11] Y. Yang, H. Zhang, S. Fan, H. Lu, M. Zhang, and X. Li, "Poseidon: Practical Homomorphic Encryption Accelerator," in *Proc. HPCA*, 2023.
- [12] Y. Zhu *et al.*, "DAHE: Parameter-Adaptive and Memory Efficient FPGA Acceleration of Homomorphic Encryption," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2687–2701, Aug. 2025.
- [13] S. Kim, K. Lee, W. Cho, J. H. Cheon, and R. A. Rutenbar, "FPGA-based Accelerators of Fully Pipelined Modular Multipliers for Homomorphic Encryption," in *Proc. ReConFig*, 2019.
- [14] S. Kim, K. Lee, W. Cho, Y. Nam, J. H. Cheon, and R. A. Rutenbar, "Hardware Architecture of a Number Theoretic Transform for a Bootstrappable RNS-based Homomorphic Encryption Scheme," in *Proc. FCCM*, 2020.
- [15] F. Krieger, F. Hirner, A. C. Mert, and S. S. Roy, "Aloha-HE: A Low-Area Hardware Accelerator for Client-Side Operations in Homomorphic Encryption," in *Proc. DATE*, 2024.
- [16] S. Yune *et al.*, "ABC-FHE: A Resource-Efficient Accelerator Enabling Bootstrappable Parameters for Client-Side Fully Homomorphic Encryption," in *Proc. DAC*, 2025.
- [17] Intel Corporation, "Intel oneAPI DPC++/C++ Compiler Handbook for Intel FPGAs," Intel documentation. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/oneapi-fpga-add-on/developer-guide/>.
- [18] D. Natarajan and W. Dai, "SEAL-Embedded: A Homomorphic Encryption Library for the Internet of Things," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2021, no. 3, pp. 756–779, 2021.
- [19] P. Longa and M. Naehrig, "Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography," in *Proc. CANS*, 2016.
- [20] P. Barrett, "Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor," in *Proc. CRYPTO*, 1986.
- [21] Microsoft, "Microsoft SEAL," [Online]. Available: <https://github.com/microsoft/SEAL>.
- [22] M. van der Hagen and B. Lucia, "Client-Optimized Algorithms and Acceleration for Encrypted Compute Offloading," in *Proc. ASPLOS*, 2022.
- [23] Z. Azad, G. Yang, R. Agrawal, D. Petrisko, M. Taylor, and A. Joshi, "RACE: RISC-V SoC for En/Decryption Acceleration on the Edge for Homomorphic Computation," in *Proc. ISLPED*, 2022.
- [24] Z. Azad, G. Yang, R. Agrawal, D. Petrisko, M. Taylor, and A. Joshi, "RISE: RISC-V SoC for En/Decryption Acceleration on the Edge for Homomorphic Encryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 10, pp. 1523–1536, Oct. 2023.
- [25] S. Di Matteo, M. Lo Gerfo, and S. Saponara, "VLSI Design and FPGA Implementation of an NTT Hardware Accelerator for Homomorphic SEAL-Embedded Library," *IEEE Access*, vol. 11, pp. 72498–72508, 2023.
- [26] A. S. Molahosseini and H. Vandierendonck, "Exploiting Data Redundancy in CKKS Encoding for High-Speed Homomorphic Encryption," in *Proc. ASIA CCS*, 2024.
- [27] A. Aikata, F. Krieger, and S. S. Roy, "Privacy at Your Fingertips: Enabling Rapid Client-Side Operations in Fully Homomorphic Encryption," in *Proc. EuroS&P*, 2026.
- [28] J. Lee, P. N. Duong, and H. Lee, "Configurable Encryption and Decryption Architectures for CKKS-Based Homomorphic Encryption," *Sensors*, vol. 23, no. 17, Art. no. 7389, Aug. 2023.
- [29] Y. Meng *et al.*, *Intel Homomorphic Encryption (HE) Acceleration Library for FPGAs (Version 2.0)*, 2022. [Online]. Available: <https://github.com/intel/hexl-fpga>.
- [30] Altera, "DSP Builder," [Online]. Available: <https://www.altera.com/products/development-tools/quartus-prime/dsp-builder>.
- [31] D. Natarajan and W. Dai, *SEAL-Embedded*, 2021. [Online]. Available: <https://github.com/microsoft/SEAL-Embedded>.